Lab - 2: A brief introduction to the various tools available in Bash

Mastering the command line is fundamental for navigating and managing files, processes, and system resources in a Bash environment. They form the basis of many tasks performed in Unix-like operating systems. Here are the most used commands in Bash, along with explanations and examples for each. Some of these commands were already covered in last week's practical handout, but it would be a good idea to practise them multiple times.

`ls`

Explanation: Lists files and directories in the current directory.

Example:

```
ls
```

This command lists files in the current directory. Like most other commands, `ls` ` has many options.

Additional Options for `ls`

You can use various options with the `ls` command to modify its output:

- **List with Detailed Information:**

  To list files with detailed information (permissions, owner, size, and modification date), use the `-l` option:

  ```
  ls -l
  ```

- **List All Files Including Hidden Files:**

  To include hidden files (those starting with a dot), use the `-a` option:

  ```
  ls -a
  ```

- **Combine Options:**

You can combine options. For example, to list all files with detailed information, you can use:

```
ls -la
```

Example Output

When you run `ls -la`, you might see output like this (assuming you have files called 'example.txt' and 'anotherfile.txt' in your directory):

```
drwxr-xr-x  2 username username 4096 Oct  1 12:00 .
drwxr-xr-x  5 username username 4096 Sep 30 11:00 ..
-rw-r--r--  1 username username  123 Oct  1 12:00 example.txt
-rw-r--r--  1 username username  456 Oct  1 12:00 anotherfile.txt
```

In this output:

- The first column shows the file type and permissions.
- The second column shows the number of links.
- The third and fourth columns show the owner and group.
- The fifth column shows the file size.
- The sixth column shows the last modification date and time.
- The last column shows the file name.

The letters `r` `w` and `x` stand for `read`, `write` and `execute` respectively. Unix-like filesystems operate on the notion of *permissions* to a file. There are three entities that are recognized: `owner`, `group` and `other`. So, in the example above, the file `example.txt` has `read` and `write` permissions for the `owner`, only `read` permissions for both `group` and `other`. If the file `example.txt` had `write` permissions for `group`, then anyone in the `student` group (all your classmates) could `write` and change the file. Right now, they can only `read` it. Check out the *man page* for `chmod` for more information on how to change these permissions.

`cd`

Explanation: Changes the current directory.

Example:

```
cd /path/to/directory
```

The `cd` (change directory) command in Bash is used to navigate between directories in the file system. Here are some examples of how to use the `cd` command.

Basic Usage

- **Change to a Specific Directory:**

If you want to change to a directory named `Documents`, you would use:

```
cd Documents
```

This command assumes that the `Documents` directory is located in your current working directory.

- **Change to a Directory Using an Absolute Path:**

If you want to change to a directory using its absolute path (the full path from the root), you can do so like this:

```
cd /home/username/Documents
```

Replace `/home/username/Documents` with the actual path to the directory you want to access.

- **Change to the Parent Directory:**

To move up one level to the parent directory, you can use:

```
cd ..
```

The `..` represents the parent directory of your current location.

- **Change to the Home Directory:**

To quickly return to your home directory, you can simply use:

```
cd \~
```

Alternatively, you can just type `cd` without any arguments:

```
cd
```

- **Change to the Previous Directory:**

If you want to switch back to the last directory you were in, you can use:

```
cd -
```

Example Scenario

Let's say your current directory structure looks like this:

```
/home/username/
├── Documents/
│ ├── file1.txt
│ └── file2.txt
├── Downloads/
│ └── [file3.zip](<http://file3.zip>)
└── Pictures/
└── image1.png
```

If you are currently in `/home/username/` and you want to navigate to the `Documents` directory, you would run:

```
cd Documents
```

Now, if you want to go back to the parent directory (which is `/home/username/`), you would run:

```
cd ..
```

If you then want to go to the `Downloads` directory using an absolute path, you would run:

```
cd /home/username/Downloads
```

Checking Your Current Directory

To check your current directory at any time, you can use the `pwd` (print working directory) command:

```
pwd
```

This will display the full path of your current directory.

---

4. `cp`

Explanation: Copies files or directories.

Example:

```
cp source.txt destination.txt
```

This command copies `source.txt` to `destination.txt`.

- **Copy a File to a Different Directory:**

If you want to copy `example.txt` to a directory named `backup`, you would use:

```
cp example.txt backup/
```

This assumes that the `backup` directory exists in the current directory.

- **Copy a Directory:**

To copy an entire directory named `myfolder` to a new directory named `myfolder_backup`, you can use the `-r` (recursive) option:

```
cp -r myfolder myfolder\_backup
```

- **Copy with Verbose Output:**

If you want to see the details of the copy operation, you can add the `-v` (verbose) option:

```
cp -v example.txt backup/
```

This will display a message indicating that the file has been copied.

Example Scenario

Let's say your current directory structure on the remote server looks like this:

```
/home/username/
├── example.txt
├── backup/
└── myfolder/
├── file1.txt
└── file2.txt
```

If you want to copy `example.txt` to the `backup` directory, you would run:

```
cp example.txt backup/
```

After running this command, the `backup` directory will now contain `example.txt`.

If you want to create a backup of `myfolder` and name it `myfolder_backup`, you would run:

```
cp -r myfolder myfolder\_backup
```

`scp`

Explanation: Copies files or directories securely across machines. You would use this, for example, to transfer files from your local machine to a remote server (such as veridian)

Example:

```
scp /path/to/local/example.txt user@veridian.ucd.ie:/home/username/
```

This command copies the file `example.txt` from the local directory on your machine (assuming that `/path/to/local/` is a valid path) to your home directory on veridian

You could also copy an entire directory named `mydir` from your local machine, to the remote server, by using the `-r` option to copy recursively.

Example:

```
scp -r /path/to/local/mydir user@veridian.ucd.ie:/home/username/
```

`mv`

Explanation: Moves or renames files and directories.

Example:

```
mv oldname.txt newname.txt
```

The `mv` command is a versatile tool and can also be used for renaming files and directories.

Here are some more detailed examples

- **Move a File to a Different Directory:**

To move a file named `example.txt` from the current directory to a directory named `backup`, you would use:

```
mv example.txt backup/
```

This command moves `example.txt` into the `backup` directory.

- **Rename a File:**

If you want to rename `example.txt` to `example_renamed.txt` in the same directory, you would use:

```
mv example.txt example\_renamed.txt
```

- **Move and Rename a File:**

You can also move a file and rename it in one command. For example, to move `example.txt` to the `backup` directory and rename it to `example_backup.txt`, you would use:

```
mv example.txt backup/example\_backup.txt
```

- **Move a Directory:**

To move a directory named `myfolder` to another directory named `archive`, you would use:

```
mv myfolder archive/
```

Example Scenario

Let's say your current directory structure on veridian looks like this:

```
/home/username/
├── example.txt
├── backup/
└── myfolder/
├── file1.txt
└── file2.txt
```

If you want to move `example.txt` into the `backup` directory, you would run:

```
mv example.txt backup/
```

After running this command, `example.txt` will no longer be in the current directory; it will be located in the `backup` directory.

If you want to rename `myfolder` to `myfolder_old`, you would run:

```
mv myfolder myfolder\_old
```

`rm`

Explanation: Removes files or directories.

Example:

```
rm file.txt
```

This command deletes `file.txt`. Use `-r` to remove directories recursively.

Here are some more examples of how to use it:

- **Remove a single file**:

```
rm file.txt
```

This command will delete the file named `file.txt` in the current directory.

- **Remove multiple files:**

```
rm file1.txt file2.txt file3.txt
```

This command will delete the files named `file1.txt`, `file2.txt`, and `file3.txt` in the current directory.

- **Remove all files in a directory**:

```
rm \*
```

This command will delete all files in the current directory. Be careful with this command, as it will delete all files without prompting for confirmation.

- **Remove a directory and its contents**:

```

```

```
rm -r dir
```

This command will delete the directory named `dir` and all of its contents. Be careful with this command, as it will delete all files and directories within `dir` without prompting for confirmation.

- **Force removal of a file without prompting for confirmation**:

```
rm -f file.txt
```

This command will delete the file named `file.txt` without prompting for confirmation.

- Interactively remove files:

```
rm -i file.txt
```

This command will prompt for confirmation before deleting the file named `file.txt`.

- Remove files that match a pattern:

```
rm \*.txt
```

This command will delete all files in the current directory that end with `.txt`.

Note: Be careful when using the `rm` command, as it will permanently delete files and directories without moving them to the trash or recycle bin.

`mkdir`

Explanation: Creates a new directory.

The `mkdir` command in Bash is used to create a new directory. Here are some examples of how to use it:

- Create a single directory:

```
mkdir new\_directory
```

This command will create a new directory named `new_directory` in the current directory.

- Create multiple directories:

```
mkdir dir1 dir2 dir3
```

This command will create three new directories named `dir1`, `dir2`, and `dir3` in the current directory.

- Create a directory with a specific path:

```
mkdir /path/to/new\_directory
```

This command will create a new directory named `new_directory` at the specified path.

- Create a directory and its parent directories if they don't exist:

```
mkdir -p /path/to/new\_directory/subdirectory
```

This command will create the `new_directory` directory and its `subdirectory` subdirectory at the specified path, even if the parent directories don't exist.

- Create a directory with specific permissions:

```
mkdir -m 755 new\_directory
```

This command will create a new directory named `new_directory` with the specified permissions (in this case, read, write, and execute permissions for the owner, and read and execute permissions for the group and others).

Note: The `mkdir` command can only create directories, not files. If you want to create a new file, use the `touch` command instead. Also, be careful when creating directories, as it can affect the file system structure and permissions.

`rmdir`

Explanation: Removes empty directories.

The `rmdir` command in Bash is used to remove an empty directory. Here are some examples of how to use it:

- Remove an empty directory:

```
rmdir old\_directory
```

This command will remove the empty directory named `old_directory` in the current directory.

- Remove multiple empty directories:

```
rmdir dir1 dir2 dir3
```

This command will remove the empty directories named `dir1`, `dir2`, and `dir3` in the current directory.

- Remove a directory with a specific path:

```
rmdir /path/to/old\_directory
```

This command will remove the empty directory named `old_directory` at the specified path.

Note: The `rmdir` command can only remove empty directories. If the directory is not empty, you will need to remove its contents first before using `rmdir`. You can use the `rm` command with the `-r` option to remove a directory and its contents recursively.

Here are some examples of how to use the `rm` command to remove a directory and its contents:

- Remove a directory and its contents:

```
rm -r old\_directory
```

This command will remove the directory named `old_directory` and its contents recursively.

- Remove a directory and its contents without prompting for confirmation:

```
rm -rf old\_directory
```

This command will remove the directory named `old_directory` and its contents recursively without prompting for confirmation. Be careful with this command, as it can permanently delete files and directories without moving them to the trash or recycle bin.

Note: Be careful when removing directories, as it can affect the file system structure and permissions. Make sure to backup any important data before removing a directory and its contents.

`echo`

Explanation: Displays a line of text or variable value.

The `echo` command in Bash is used to display a message or the value of a variable on the terminal. Here are some examples of how to use it:

- Display a message:

```
echo "Hello, World!"
```

This command will display the message "Hello, World!" on the terminal.

- Display the value of a variable:

```
name="John Doe"
echo $name
```

This command will display the value of the `name` variable, which is "John Doe".

- Concatenate and display multiple messages:

```
first\_name="John"
last\_name="Doe"
echo "$first\_name $last\_name"
```

This command will display the concatenated message "John Doe" on the terminal.

- Display the output of a command:

```
echo "The current date is $(date)"
```

This command will display the message "The current date is" followed by the output of the `date` command.

- Redirect the output of a command to a file:

```
echo "Hello, World!" > greetings.txt
```

This command will redirect the output of the `echo` command to a file named `greetings.txt`. If the file already exists, it will be overwritten.

- Append the output of a command to a file:

```
echo "Hello, World!" >> greetings.txt
```

This command will append the output of the `echo` command to a file named `greetings.txt`. If the file does not exist, it will be created.

We will cover redirection in more detail in the next practical session. For the moment, simply think of the 'pointy end' of the symbol '>' as indicating *where* the output will go.

`man`

Explanation: Displays the manual pages for other commands.

Example:

```
man ls
```

This command opens the manual page for the `ls` command. This is a great way to get detailed information about how to use a command, including its options, syntax, and examples.

Example Usage of `man`

Getting Help for a Specific Command:

For example, if you want to get information about the `ls` command, you would use:

```
man ls
```

This command will open the manual page for `ls`, displaying information about its usage, options, and examples.

Navigating the Manual Page:

Once you are in the manual page, you can navigate using the following keys:

- Use the **Up** and **Down** arrow keys to scroll through the content.
- Press **Space** to scroll down one page.
- Press **b** to scroll up one page.
- Press **/`** followed by a search term to search for specific text within the manual.
- Press **q** to quit and exit the manual page.

Getting Help for Other Commands:

You can use `man` with any command. For example, to get information about the `cp` command, you would run:

```
man cp
```

Or for the `mv` command:

```
man mv
```

Example Output

When you run `man ls`, you might see output similar to this:

```
LS(1) User Commands LS(1)

NAME
ls - list directory contents

SYNOPSIS
ls [OPTION]... [FILE]...

DESCRIPTION
List information about the FILEs (the current directory by default).

-a, --all
do not ignore entries starting with .

-l use a long listing format

-h, --human-readable
with -l, print sizes in human readable format (e.g., 1K 234M 2G)

...

EXAMPLES
ls -l
List files in long format.

ls -a
List all files, including hidden files.

SEE ALSO
The full documentation for ls is maintained as a Texinfo manual. If the inf

info coreutils 'ls invocation'
```

```
should give you access to the complete manual.
```

The `man` command is a powerful tool for accessing the documentation of commands in Unix-like systems. It provides detailed information about command usage, options, and examples, making it an essential resource for users looking to understand how to use various commands effectively.

Now that you know how to use `man`, we will merely concentrate on listing commands with simple examples. You are expected to navigate the multiple tools and their options by looking at the 'man pages'.

`cat`

Explanation: Concatenates and displays the content of files.

Example:

```
cat file.txt
```

This command displays the content of `file.txt`.

`touch`

Explanation: Creates an empty file or updates the timestamp of an existing file.

Example:

```
touch newfile.txt
```

This command creates an empty file named `newfile.txt`.

`grep`

Explanation: Searches for a specific pattern in files.

Example:

```
grep "search\_term" file.txt
```

This command searches for `search_term` in `file.txt`.

`find`

Explanation: Searches for files and directories in a directory hierarchy.

Example:

```
find /path/to/search -name "\*.txt"
```

This command finds all `.txt` files in the specified path.

`grep` uses special symbols for 'globbing'. We will cover this in more detail in a later session. For the moment, think of '*' as a special character that matches anything.

`chmod`

Explanation: Changes the permissions of a file or directory.

Example:

```
chmod 755 [script.sh](<http://script.sh>)
```

This command sets the permissions of `script.sh` to read, write, and execute for the owner, and read and execute for others.

`chown`

Explanation: Changes the owner of a file or directory.

Example:

```
chown user:group file.txt
```

This command changes the owner of `file.txt` to `user` and the group to `group`.

`ps`

Explanation: Displays information about running processes.

Example:

```
ps aux
```

This command shows detailed information about all running processes.

`kill`

Explanation: Sends a signal to terminate a process.

Example:

```
kill 1234
```

This command sends a termination signal to the process with PID 1234.

`top`

Explanation: Displays real-time information about system processes and resource usage.

Example:

```
top
```

This command opens an interactive display of system processes.

`df`

Explanation: Displays disk space usage for file systems.

Example:

```
df -h
```

This command shows disk space usage in a human-readable format.

`du`

Explanation: Displays disk usage of files and directories.

Example:

```
du -sh /path/to/directory
```

This command shows the total disk usage of the specified directory.

`history`

Explanation: Displays the command history.

Example:

```
history
```

This command lists all previously executed commands.

`alias`

Explanation: Creates shortcuts for commands.

Example:

```
alias ll='ls -l'
```

This command creates an alias `ll` for `ls -l`.

`wget`

Explanation: Downloads files from the web.

Example:

```
wget [http://example.com/file.zip](<http://example.com/file.zip>)
```

This command downloads `file.zip` from the specified URL.

Strictly speaking, `wget` is not a part of the commandset of Bash. However, it is frequently used by system administrators to download files from various sources. It worthwhile reading through the various options that `wget` provides and becoming familiar with them. Another tool that is frequently used is `curl`.
